

ITE5231: DATA STRUCTURES & DESIGN PATTERNS

Hospital Patient Management System

Group Project 1 Report

Professor

Dr. Syed Raza Bashir

Group Members

Tu Lien Dang (N01704642)

Jan Brian Ano-os (N10003819)

Table of Contents

1. INTRODUCTION.....	1
2. SYSTEM ARCHITECTURE.....	1
3. DATA STRUCTURES AND WHY THEY WERE CHOSEN	2
3.1 Dictionary — patient registration.....	2
3.2 Priority queue built as a sorted linked list — triage.....	3
3.3 Stack — appointments per doctor	3
3.4 Linked list — medical history	3
3.5 Queue — notification dispatch.....	4
4. DESIGN PATTERNS AND WHY THEY WERE CHOSEN	4
4.1 Singleton — HospitalSystem.....	4
4.2 Iterator — MedicalHistoryIterator	4
4.3 Observer — PatientStatusNotifier.....	5
4.4 Strategy — IBillingStrategy	5
5. Payments, Receipts and Appointment Validation	5
6. Time Complexity.....	6
7. Testing	7
8. Development Process	7
9. Conclusion	8

1. INTRODUCTION

This report describes the design and implementation of a Hospital Patient Management System, a console application written in C# on .NET 8. The system carries out the everyday tasks of a small hospital: registering patients, ordering them for treatment according to how urgent their condition is, scheduling and cancelling appointments with doctors, keeping a medical history for each patient, informing staff whenever a patient's status changes, and calculating and collecting payment.

The aim of the project was to apply core data structures and object-oriented design patterns to a realistic problem rather than to isolated exercises. Six data structures are used: a dictionary, a priority queue, a sorted linked list, a stack, a linked list, and a queue. Four Gang-of-Four design patterns are used: Singleton, Iterator, Observer, and Strategy. This report explains each choice and, more importantly, why that choice suits the feature it supports.

The work was completed in two phases. Phase 1 built the data-management foundation: the domain models, the four hand-written or built-in structures behind registration, triage, appointments and medical history, and the Singleton and Iterator patterns. Phase 2 added status notifications, billing, payments, appointment validation and the interactive console workflow, together with the Observer and Strategy patterns. Every source file carries a comment header marking it as Phase 1 or Phase 2, and additions made to Phase 1 files during Phase 2 are wrapped in matching comment markers so the two stages of work can be told apart.

2. SYSTEM ARCHITECTURE

The solution is divided into two projects: the application itself and a separate xUnit test project. Within the application, code is grouped by responsibility rather than by feature, which keeps each folder small and gives each class one clear job.

- Models — plain domain objects and enumerations: Patient, Appointment, MedicalRecord, Payment, PatientAccount, TriageLevel, PatientStatus and PaymentMethod.
- DataStructures — the structures written by hand for this project: TriageQueue, MedicalHistory and MedicalHistoryIterator.
- Services — PatientRegistry, AppointmentService and HospitalSystem, which coordinates the others.
- Notifications — the Observer implementation: the notifier and its three observers.
- Billing — the Strategy implementation: the billing interface and its three algorithms.
- UI — ConsoleMenu, the menu-driven workflow, together with Program.cs as the entry point.

HospitalSystem acts as a single front door to the system. It owns the registry, the triage queue, the appointment service, the notifier and the current billing strategy, so no other class needs to assemble those parts itself. The user interface is deliberately confined to one class. ConsoleMenu reads and writes through TextReader and TextWriter rather than calling Console directly, which means the entire workflow can be driven by an automated test supplying scripted input.

Figure 1 shows the class diagram for the completed system.

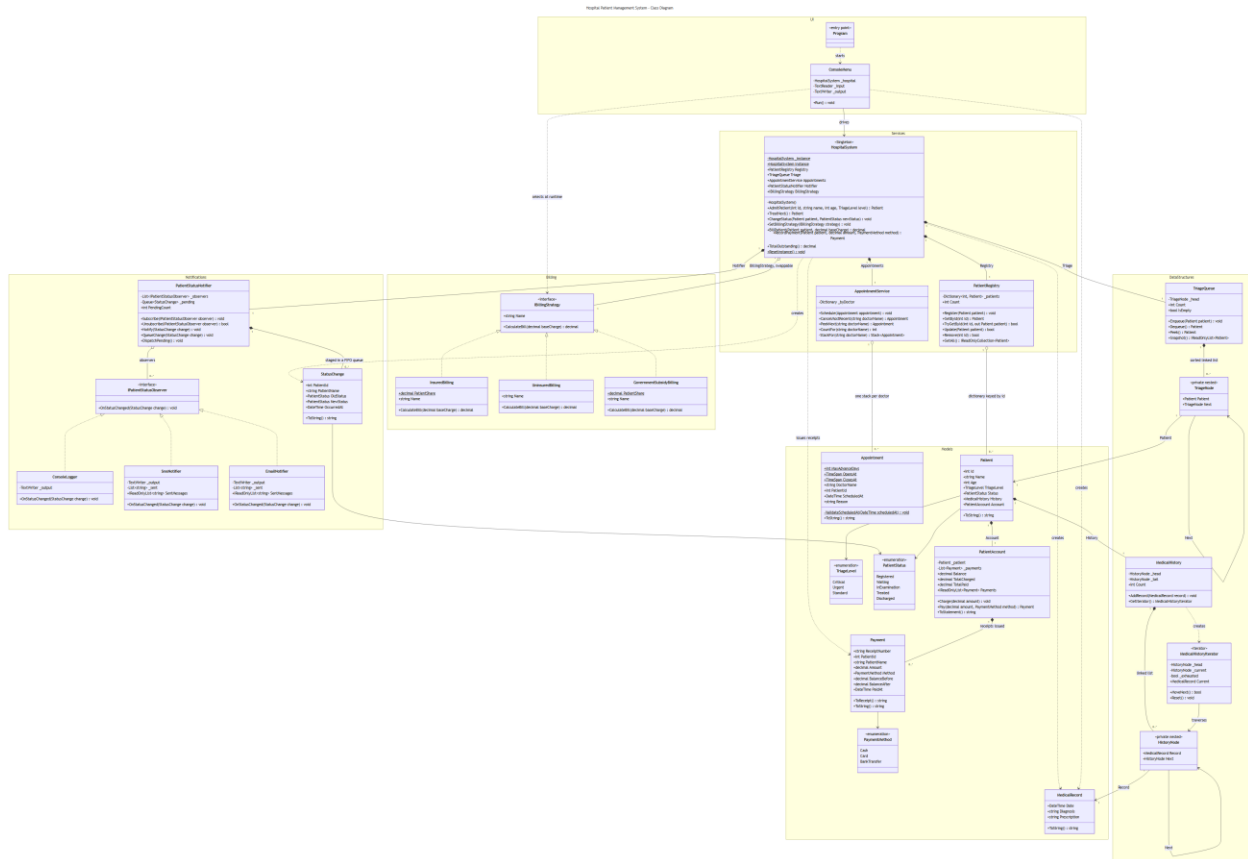


Figure 1. UML class diagram. A full-resolution copy is supplied as docs/uml-class-diagram.png.

3. DATA STRUCTURES AND WHY THEY WERE CHOSEN

3.1 Dictionary — patient registration

Patient records are held in a Dictionary<int, Patient> inside PatientRegistry, keyed on the patient's unique identifier. The registry supports adding, searching, updating and deleting a record.

The operation that dominates this feature is "find the patient with this ID", which happens on almost every menu action. A dictionary hashes the key directly to a storage bucket, so lookup, insertion and removal all take constant time on average, no matter how many patients are registered. Storing the same records in a list or array would mean scanning through the entries one by one until the right ID appeared, which takes time proportional to the number of patients. Patient IDs are also a natural fit for the key: they are unique by definition, and integers are cheap to hash and rarely collide. Duplicate registration is prevented by using TryAdd, which reports failure rather than silently overwriting an existing patient.

3.2 Priority queue built as a sorted linked list — triage

TriageQueue is a singly linked list that is kept in order of triage level at all times: Critical (1) first, then Urgent (2), then Standard (3). It satisfies two of the required structures at once, because the priority queue is implemented by keeping the linked list sorted.

Triage is by nature a priority problem rather than a first-come-first-served one: a patient in a critical condition must be treated before someone with a minor complaint who arrived earlier. Sorting the list as each patient is inserted means the most urgent patient is always sitting at the head, so removing or inspecting the next patient takes constant time. Insertion costs time proportional to the queue length, because the new node has to be walked into the correct position. This is a deliberate trade-off. A patient is admitted once, but the queue is examined and drained repeatedly as doctors become free, so it is better to pay the cost on the rarer operation and keep the frequent one fast.

Order is also preserved among patients of equal severity. A new node is inserted after all existing nodes of the same triage level, so two Critical patients are seen in the order they arrived. This first-in-first-out behaviour within a priority level matters clinically and is covered by its own test.

3.3 Stack — appointments per doctor

Each doctor has an independent Stack<Appointment>, and the stacks are held in a dictionary keyed on the doctor's name. Scheduling pushes an appointment; cancelling pops the most recent one.

The behaviour the requirement asks for is an undo: cancel the last appointment that was scheduled. Last-in-first-out is the defining property of a stack, so the structure matches the requirement exactly rather than merely being capable of supporting it. Push, pop and peek are all constant-time operations. Combining the stacks with a dictionary means each doctor keeps a separate undo history, and locating the correct stack is itself a constant-time lookup. It is worth acknowledging the limitation: a stack is a poor choice for finding an appointment by date or patient, since that would require examining every entry. The specified behaviour, however, is undo, and for undo a stack is the natural structure.

3.4 Linked list — medical history

Every patient owns a MedicalHistory, a singly linked list of MedicalRecord nodes holding a date, a diagnosis and a prescription. Records are stored oldest first.

A medical history grows one record at a time and is almost always read from beginning to end. A linked list suits both habits. Adding a node simply links it into the chain, with no resizing or copying of existing data, whereas an array-backed list must occasionally allocate a larger array and copy everything across. Keeping the list ordered by date means traversal is chronological by construction, so no sorting step is needed before a history is displayed. A tail pointer is kept so that the ordinary case, a new record dated today and therefore later than everything already stored, is added in constant time; only a back-dated record has to walk the list to find its position. The usual weakness of a linked list, slow access to an arbitrary position, does not apply here, because histories are read in sequence and records are never deleted.

3.5 Queue — notification dispatch

PatientStatusNotifier stages each status change in a Queue<StatusChange> before delivering it to the registered observers.

Notifications must reach observers in the order the events actually occurred. A patient moving from Waiting to InExamination and then to Treated has to be reported in that sequence, or the resulting log misrepresents what happened to the patient. A queue is first-in-first-out by definition, so it preserves that ordering without any additional work. Using a queue also separates recording an event from delivering it: QueueChange adds a change without dispatching it, and DispatchPending drains the queue and broadcasts every entry. Both enqueue and dequeue are constant-time operations.

4. DESIGN PATTERNS AND WHY THEY WERE CHOSEN

4.1 Singleton — HospitalSystem

HospitalSystem has a private constructor and exposes a single shared instance through a static property. It owns the registry, the triage queue, the appointment service, the notifier and the current billing strategy.

The problem this solves is consistency. These collections must be shared across the whole program: if the menu created one registry and the billing code created another, a patient registered through the menu would be invisible when the bill was calculated. There is conceptually exactly one hospital in this application, and its data must be the same everywhere it is read. Passing references to every collection into every class would also work, but each class would then have to accept and forward objects it does not use. The Singleton gives one obvious, unambiguous point of access instead.

The drawback should be stated honestly: a singleton is global mutable state, which can make testing awkward because one test can leave data behind that affects the next. Two measures address this. A ResetInstance method replaces the instance with a fresh one, and each test class calls it before running so every test starts from an empty hospital. Test parallelisation is also switched off at assembly level, because the shared instance cannot safely be used by several tests at once.

4.2 Iterator — MedicalHistoryIterator

MedicalHistoryIterator walks a patient's history from the oldest record to the newest, exposing MoveNext, Current and Reset.

The Iterator pattern separates the job of traversing a collection from the job of storing it. Code that prints a medical history uses a simple loop and never sees a node or a Next pointer; indeed the node class is private to MedicalHistory and cannot be reached from outside. This matters for maintenance: if the history were later reorganised as an array or a tree, the iterator would change but every caller would keep working unaltered. It also prevents outside code from corrupting the list by rearranging links directly.

One detail is worth recording. Once MoveNext has returned false the iterator remains exhausted until Reset is called explicitly. An earlier version tracked only the current node, which meant that reaching the end left

the iterator in the same state as before it started, and a further call quietly restarted from the first record. Any caller that did not stop at the first false would loop forever. The corrected version keeps a separate flag for exhaustion, and a test now checks that repeated calls after the end continue to return false.

4.3 Observer — PatientStatusNotifier

When a patient's status changes, three things must happen: the change is written to a console log, an SMS is sent, and an email is sent. PatientStatusNotifier is the subject; ConsoleLogger, SmsNotifier and EmailNotifier each implement IPatientStatusObserver and subscribe to it.

Without this pattern, HospitalSystem would have to call each notifier itself, which creates two problems. It would need to know about every notification channel that exists, and adding a fourth channel would mean editing HospitalSystem even though nothing about admitting or treating a patient had changed. With the Observer pattern, HospitalSystem announces the change once and the notifier passes it to whoever is listening. A new channel is added by writing a class and subscribing it, leaving existing code untouched, which is what the open/closed principle recommends. The three observers are also independent of one another: none knows the others exist, so one can be removed without disturbing the rest. The SMS and email observers are stubs, as no real gateway is available, but each records the message it would have sent so that tests can confirm the correct notification was produced.

4.4 Strategy — IBillingStrategy

A bill can be calculated in three ways: an insured patient pays twenty per cent of the base charge, an uninsured patient pays the full amount, and a patient on a government subsidy pays forty per cent. Each rule is a small class implementing IBillingStrategy, and HospitalSystem holds whichever one is currently selected.

The obvious alternative is a conditional statement inside the billing method that checks what kind of patient is being billed. That approach deteriorates quickly: the method grows every time a new arrangement is introduced, all the pricing rules become entangled in one place, and testing one rule means working around the others. The Strategy pattern gives each rule its own class that can be read, tested and modified on its own. Introducing a fourth billing arrangement means adding a class, not editing code that already works. Because the strategy is stored as an interface, it can be exchanged while the program is running, which is what the requirement asks for: the user picks a billing method from the menu and every subsequent bill uses it. Each of the three implementations has its own unit tests, and a further test confirms that swapping the strategy changes the amount charged.

5. Payments, Receipts and Appointment Validation

Each patient owns a PatientAccount recording the outstanding balance, the total charged, the total paid, and every receipt issued. Billing a patient prices the base charge with the current strategy, adds the result to the balance, and notes it in the patient's history. Recording a payment settles part or all of the balance

and returns a Payment object, which acts as the receipt: it carries a receipt number, a timestamp, the payment method, the amount, and the balance both before and after.

Receipt numbers follow the form RCP-0101-002, built from the patient's identifier and the number of payments already made on that account. Deriving the number this way means no shared counter is needed anywhere in the system, yet the number is still unique, because patient identifiers are themselves unique. Overpayment is rejected rather than accepted, so a balance can never become negative; partial payments are supported, which is the normal case in practice.

Appointment dates are validated inside the Appointment constructor. A date must be in the future, no more than one year ahead, and between 08:00 and 18:00. Placing these checks in the constructor rather than in the menu means an invalid appointment cannot be created at all, whichever code path attempts it, so no caller can forget to validate. The boundaries are treated deliberately: 08:00 is accepted and 18:00 is rejected, since an appointment starting exactly at closing time cannot take place.

6. Time Complexity

The table below summarizes the cost of each operation, where n is the number of items held in the structure concerned.

Structure	Class	Operation	Complexity
Dictionary	PatientRegistry	Register / GetById / Update / Remove	$O(1)$ average
Dictionary	PatientRegistry	GetAll	$O(n)$
Priority queue (sorted linked list)	TriageQueue	Enqueue (sorted insert)	$O(n)$
	TriageQueue	Dequeue / Peek	$O(1)$
	TriageQueue	Snapshot	$O(n)$
Stack	AppointmentService	Schedule / CancelMostRecent / PeekNext	$O(1)$
Linked list	MedicalHistory	AddRecord (newest record)	$O(1)$ via tail pointer
	MedicalHistory	AddRecord (back-dated)	$O(n)$
Queue	PatientStatusNotifier	QueueChange / dispatch one change	$O(1)$
Iterator	MedicalHistoryIterator	MoveNext / Current / Reset	$O(1)$
Account	PatientAccount	Charge / Pay	$O(1)$

The one deliberate compromise is TriageQueue, which pays $O(n)$ on insertion so that removal can be $O(1)$. A binary heap would make both operations $O(\log n)$ and would be preferable for a very large queue, but the specification requires a sorted linked list, and for a waiting room of realistic size the linear insert is not a practical concern.

7. Testing

The system is covered by 60 tests, spread across ten test classes. All 60 pass. The requirement was a minimum of ten test cases.

Test class	Tests	Area covered
PatientRegistryTests	6	Dictionary add, search, update, delete, duplicates
TriageQueueTests	4	Priority ordering and FIFO among equal levels
AppointmentServiceTests	4	LIFO cancellation, independent stacks per doctor
MedicalHistoryTests	10	Linked list insertion, iterator traversal and reset
HospitalSystemTests	6	Singleton, admission, treatment, billing, balances
StatusNotifierTests	5	Observer subscribe, unsubscribe, FIFO dispatch
BillingStrategyTests	5	Each billing algorithm and runtime swapping
PatientAccountTests	11	Charges, payments, receipts, statements
AppointmentValidationTests	7	Past dates, advance limit, clinic hours
WorkflowTests	2	End-to-end menu workflow with scripted input

Tests were written for behaviour rather than for line coverage, so the awkward cases are covered explicitly: two patients of equal triage level leaving the queue in arrival order, an iterator that has run past the end, a payment larger than the outstanding balance, and an appointment booked at exactly the opening and closing times. WorkflowTests is an integration test rather than a unit test; it drives the complete console menu with a scripted sequence of inputs and checks the resulting output, which is possible only because the menu reads and writes through TextReader and TextWriter.

8. Development Process

Work was tracked on a GitHub Projects board with one issue per requirement, each carrying a priority and an estimate and assigned to one of the two sprints. Commit messages reference their issue number so that the commits appear against the issue, and pull requests close their issues automatically on merge.

The two-phase split is recorded in the code itself. Each file begins with a comment marking it as Phase 1 or Phase 2, and where Phase 2 work was added to a file created in Phase 1, the addition is enclosed in matching start and end comment markers. Four files are annotated this way, including Patient, Appointment

and HospitalSystem. This makes the boundary between the two stages visible without consulting the commit history.

9. Conclusion

The completed system meets the functional requirements set out for the project. All six required data structures are used, and each was chosen because its defining property matches the behaviour the feature needs: a dictionary for lookup by identifier, a sorted linked list for priority ordering, a stack for undo, a linked list for an append-and-read history, and a queue for ordered notification delivery. All four required design patterns are applied where they solve a real problem in the design rather than being added for their own sake, and in each case the report has set out the alternative that was rejected and why.

The main lesson from the work was that choosing a structure means choosing which operation to make fast. The triage queue is the clearest example: sorting on insertion is slower when a patient arrives but makes treating the next patient immediate, which is the right balance because patients are admitted once and the queue is drained repeatedly. The iterator defect found during Phase 2 was a second useful lesson, in that the original code passed every test written for it while still being wrong for a case nobody had thought to test.